Minggu Ke 1
Requirement Analysis

Copyright © March 2026

KONTRAK PERKULIAHAN

1. Mata kuliah Software Quality Assurance menggunakan metode pembelajaran *case study, Discovery Learning, and Project base learning*
2. Aktifitas pembelajaran telah tertuang dalam rencana pembelajaran semester (RPS), Rincian tugas dan rubrik penilaian tertuang pada rencana tugas mahasiswa (RTM)
3. Mahasiswa membentuk kelompok yang sama untuk semua matakuliah di semester 3

2. Masing-masing kelompok berjumlah 3-5 atau menyesuaikan jumlah setiap kelas. Kelompok dapat dibentuk pada minggu ke 1 atau 2 dan dapat diajukan pada minggu ke 3

3. Tidak ada pelaksanaan UTS dan UAS

Copyright © March 2026

KONTRAK PERKULIAHAN

4. Aktifitas pembelajaran:

No	Aktifitas Pembelajaran	Keterangan
1	Minggu ke 1 – 3 Materi	Referensi untuk mengetahui bentuk penugasan terstruktur, belajar mandiri termasuk didalamnya menyampaikan progres proyek akhir dapat menggunakan RTM
2	Minggu ke 4 Pembahasan proyek akhir	
3	Minggu ke 5 – 7 Materi	
4	Minggu ke 8 Report kemajuan proyek akhir dan penilaian anggota kelompok.	
5	Minggu ke 9-10 Bimbingan Proyek Akhir	
6	Minggu ke 11-15 Laporan dan presentasi proyek akhir	
7	Minggu 16 Entry nilai akhir	

Copyright © March 2026

Software Testing Life Cycle (STLC)

STLC atau Software Testing Life Cycle adalah serangkaian langkah atau tahapan yang dilalui dalam proses pengujian perangkat lunak (software testing) untuk memastikan kualitas perangkat lunak tersebut. Tujuan dari STLC adalah untuk mendeteksi kesalahan (bug) atau kekurangan dalam perangkat lunak, serta memastikan bahwa perangkat lunak tersebut memenuhi persyaratan yang ditetapkan oleh pengguna atau klien

Proses pengujian perangkat lunak tidak hanya melibatkan pencarian bug, tetapi juga memastikan bahwa perangkat lunak tersebut berfungsi dengan baik, memenuhi kebutuhan pengguna, dan dapat digunakan dengan aman dan efisien. Dengan mengikuti STLC, pengujian perangkat lunak menjadi lebih terstruktur dan dapat memberikan hasil yang lebih baik

Copyright © March 2026

Tahapan-Tahapan dalam STLC

1. **Pengumpulan Kebutuhan (Requirement Gathering)**

STLC adalah pengumpulan kebutuhan, yang diatur dalam **Dokumen Spesifikasi**. Di tahap ini, penting untuk mendapatkan persetujuan dari klien mengenai dokumen kebutuhan untuk memastikan kebenaran spesifikasi yang diinginkan klien. Testers akan menggunakan kebutuhan ini untuk merancang **test case**.

<u>Aktivitas pada Tahap Ini:</u>

• Menyusun Requirement Traceability Matrix (RTM) yang menghubungkan kebutuhan dengan pengujian.

• Memastikan kelayakan otomatisasi pengujian

Copyright © March 2026

Tahapan-Tahapan dalam STLC (lanjutan)

2. **Perencanaan Pengujian (Test Planning)**

   Di tahap ini, **Test Lead** atau **Manajer** akan membuat dokumen **Test Plan** yang berfungsi sebagai referensi untuk semua aktivitas pengujian.

Isi dari Test Plan:

- **Entry Criteria:** Kondisi yang harus dipenuhi agar pengujian dapat dimulai.
- **Scope of Testing:** Daftar kebutuhan yang akan diuji dalam rilis tersebut.
- **Out of Scope:** Kebutuhan yang tidak diuji dalam rilis ini.
- **Assumptions:** Asumsi-asumsi yang diterima untuk pengujian.
- **Jadwal:** Jadwal siklus pengujian dan estimasi waktu yang dibutuhkan.
- **Defect Management:** Proses pelaporan dan penutupan defect.

Copyright © March 2026

Tahapan-Tahapan dalam STLC (lanjutan)

3. Pengembangan Test Case (Test Case Development)

Test case adalah sekumpulan langkah yang digunakan tester untuk memverifikasi apakah sistem yang diuji memenuhi persyaratan yang ditentukan dan berfungsi dengan benar. Test case biasanya ditulis berdasarkan Dokumen Kebutuhan (SRS).

Jenis-jenis Test Case:

- Test Case Fungsional: Untuk memverifikasi aspek fungsional perangkat lunak.
- Test Case Non-Fungsional: Untuk memverifikasi elemen-elemen yang tidak terkait langsung dengan fungsionalitas, seperti layout halaman, ukuran gambar, dll.
- Test Case Integrasi: Untuk memverifikasi interaksi antara sistem.
- Test Case Kinerja: Untuk menguji performa aplikasi, seperti waktu muat halaman

Copyright © March 2026

Tahapan-Tahapan dalam STLC (lanjutan)

4. Penyiapan Lingkungan Pengujian (Test Environment Setup)

Di tahap ini, lingkungan pengujian harus disiapkan. Biasanya, pengujian dilakukan pada lingkungan yang terpisah seperti TEST, INT, dan PROD (produk). Setiap lingkungan memiliki pengaturan khusus sesuai kebutuhan pengujian

5. Eksekusi Test Case (Test Case Execution)

Setelah lingkungan pengujian siap, pengujian dimulai dengan menjalankan test case yang telah dikembangkan. Setiap tester menjalankan sekumpulan test case dan mencatat hasilnya. Jika terjadi kegagalan, tester harus membuka defect dan melaporkannya. Pengujian dilanjutkan sampai seluruh test case dijalankan

Copyright © March 2026

Tahapan-Tahapan dalam STLC (lanjutan)

6. Penutupan Pengujian/Maintenance (Test Cycle Closure/Maintenance)

Setelah semua test case dijalankan dan defect yang ditemukan telah ditangani, proses pengujian dinyatakan selesai. Semua deliverables dari pengujian diserahkan kepada tim pengembangan dan manajer proyek.

Deliverables pada Tahap Ini:

- Ringkasan hasil pengujian
- Daftar defect
- Sertifikasi apakah build tersebut siap untuk diteruskan ke tahap berikutnya.

Copyright © March 2026

Software Testing Life Cycle (STLC)

Sumber: https://www.testbytes.net/blog/software-testing-life-cycle/

Copyright © March 2026

Penjelasan Gambar Tahapan-Tahapan dalam STLC

1. Requirements (Persyaratan)

Tahap pertama adalah Requirements atau Persyaratan. Pada tahap ini, pengujian dimulai dengan memahami kebutuhan pengguna atau klien. Tim pengujian akan mendapatkan dokumen yang berisi semua persyaratan yang diperlukan untuk perangkat lunak tersebut. Persyaratan ini bisa berupa dokumen spesifikasi atau user stories (cerita pengguna), tergantung apakah proyek menggunakan pendekatan Agile atau tradisional.

Tujuan: Memahami fitur dan fungsionalitas yang perlu diuji agar perangkat lunak dapat berfungsi sesuai yang diinginkan.

Hasil: Dokumen persyaratan yang jelas, yang menjadi dasar untuk langkah-langkah pengujian selanjutnya

Copyright © March 2026

Penjelasan Gambar Tahapan-Tahapan dalam STLC

2. Test Plan (Rencana Pengujian)

Setelah persyaratan dikumpulkan, tim pengujian membuat Test Plan atau Rencana Pengujian. Rencana ini mencakup strategi pengujian, ruang lingkup pengujian, waktu yang dibutuhkan, dan siapa saja yang terlibat dalam proses pengujian.

Tujuan: Menentukan bagaimana pengujian akan dilakukan dan memastikan bahwa semua tim memiliki pemahaman yang sama.

Hasil: Dokumen rencana pengujian yang jelas, termasuk jadwal dan tanggung jawab.

Copyright © March 2026

Penjelasan Gambar Tahapan-Tahapan dalam STLC

3. Test Design & Review (Desain Pengujian & Tinjauan)

Pada tahap ini, tim pengujian merancang test cases atau kasus uji. Kasus uji adalah langkah-langkah yang akan diikuti untuk memeriksa apakah perangkat lunak berfungsi seperti yang diinginkan. Setelah desain kasus uji dibuat, mereka akan ditinjau oleh tim untuk memastikan bahwa tidak ada aspek yang terlewatkan.

Tujuan: Membuat kasus uji yang lengkap dan memadai untuk menguji seluruh fungsionalitas perangkat lunak.

Hasil: Daftar lengkap kasus uji yang akan digunakan untuk pengujian.

Copyright © March 2026

Penjelasan Gambar Tahapan-Tahapan dalam STLC

4. Test Case Preparation (Persiapan Kasus Uji)

Setelah desain selesai, tim pengujian mempersiapkan test case secara rinci. Pada tahap ini, mereka memastikan bahwa semua input, langkah-langkah pengujian, dan hasil yang diharapkan telah disiapkan dengan benar.

Tujuan: Memastikan bahwa setiap langkah pengujian jelas dan dapat diikuti dengan mudah oleh penguji.

Hasil: Kasus uji yang siap untuk diimplementasikan dalam pengujian.

Copyright © March 2026

Penjelasan Gambar Tahapan-Tahapan dalam STLC

5. Test Execution (Eksekusi Pengujian)

Tahap ini adalah tahap di mana pengujian benar-benar dilaksanakan. Tim pengujian menjalankan kasus uji yang telah disiapkan dan memeriksa apakah perangkat lunak berfungsi dengan baik sesuai dengan persyaratan. Jika ditemukan masalah, penguji akan mendokumentasikannya dalam Test/Bug Report.

Tujuan: Menguji perangkat lunak dan memverifikasi apakah fitur bekerja sesuai yang diinginkan.

Hasil: Laporan pengujian yang mencakup hasil setiap kasus uji (lulus atau gagal) dan masalah yang ditemukan.

Copyright © March 2026

Penjelasan Gambar Tahapan-Tahapan dalam STLC

6. Test/Bug Report (Laporan Pengujian/Bug)

Setelah eksekusi pengujian, tahap berikutnya adalah membuat Test/Bug Report. Jika ditemukan bug atau masalah selama pengujian, tim pengujian akan mencatatnya dalam laporan ini. Laporan ini akan mencakup deskripsi masalah, langkah-langkah untuk mereproduksi bug, dan tingkat keparahan bug tersebut.

Tujuan: Melaporkan semua bug dan masalah yang ditemukan selama pengujian kepada tim pengembangan agar dapat diperbaiki.

Hasil: Laporan bug yang lengkap yang memberikan gambaran tentang masalah yang ditemukan selama pengujian.

Copyright © March 2026

Penjelasan Gambar Tahapan-Tahapan dalam STLC

7. Bug Fixes/Regression Test (Perbaikan Bug/Uji Regresi)

Setelah bug dilaporkan, tim pengembangan akan memperbaikinya. Setelah perbaikan dilakukan, pengujian regresi akan dilakukan untuk memastikan bahwa perbaikan bug tidak merusak fitur lain yang sudah ada dalam perangkat lunak.

Tujuan: Memperbaiki bug yang ditemukan dan memastikan perbaikan tidak memengaruhi fungsi perangkat lunak lainnya.

Hasil: Perbaikan bug dan pengujian ulang untuk memastikan semuanya berfungsi dengan baik.

Copyright © March 2026

Penjelasan Gambar Tahapan-Tahapan dalam STLC

8. Software Release (Rilis Perangkat Lunak)

Setelah semua pengujian selesai dan perangkat lunak telah diperbaiki, tahap terakhir adalah Software Release atau Rilis Perangkat Lunak. Pada tahap ini, perangkat lunak siap untuk dirilis kepada pengguna akhir.

Tujuan: Mendistribusikan perangkat lunak yang sudah diuji dan diperbaiki kepada pengguna atau klien.

Hasil: Perangkat lunak yang siap digunakan oleh pengguna dengan keyakinan bahwa kualitasnya telah dipastikan.

Copyright © March 2026

Jenis-Jenis Tes dalam STLC

Selain tahapan-tahapan di atas, ada juga beberapa jenis pengujian yang sering dilakukan dalam STLC, yaitu:

- Pengujian Fungsional (Functional Testing): Memeriksa apakah semua fitur perangkat lunak bekerja seperti yang diinginkan.
- Pengujian Non-Fungsional (Non-Functional Testing): Menguji aspek non-fungsional perangkat lunak, seperti kinerja dan keamanan.
- Pengujian Integrasi (Integration Testing): Memeriksa bagaimana berbagai bagian perangkat lunak bekerja bersama-sama.
- Pengujian Kinerja (Performance Testing): Mengukur kinerja perangkat lunak di bawah berbagai kondisi.
- Pengujian Keamanan (Security Testing): Memastikan perangkat lunak aman dari potensi ancaman dan kerentanannya

Copyright © March 2026

Definisi Requirement Analysis

Requirement Analysis adalah proses sistematis untuk mengidentifikasi, mendokumentasikan, dan memvalidasi kebutuhan pengguna dan sistem dalam pengembangan perangkat lunak. Analisis kebutuhan berfungsi sebagai fondasi dalam Software Quality Assurance (SQA), karena kesalahan dalam tahap ini dapat berdampak besar pada kualitas perangkat lunak yang dikembangkan.

Peran Requirement Analysis dalam SQA

Requirement Analysis memiliki peran penting dalam SQA karena:

- Membantu memastikan bahwa sistem yang dikembangkan sesuai dengan kebutuhan pengguna dan pemangku kepentingan.
- Mengurangi risiko perubahan kebutuhan di tengah proyek, yang dapat memengaruhi biaya dan jadwal.
- Menjadi dasar untuk perancangan pengujian perangkat lunak, baik dalam pengujian fungsional maupun non-fungsional.

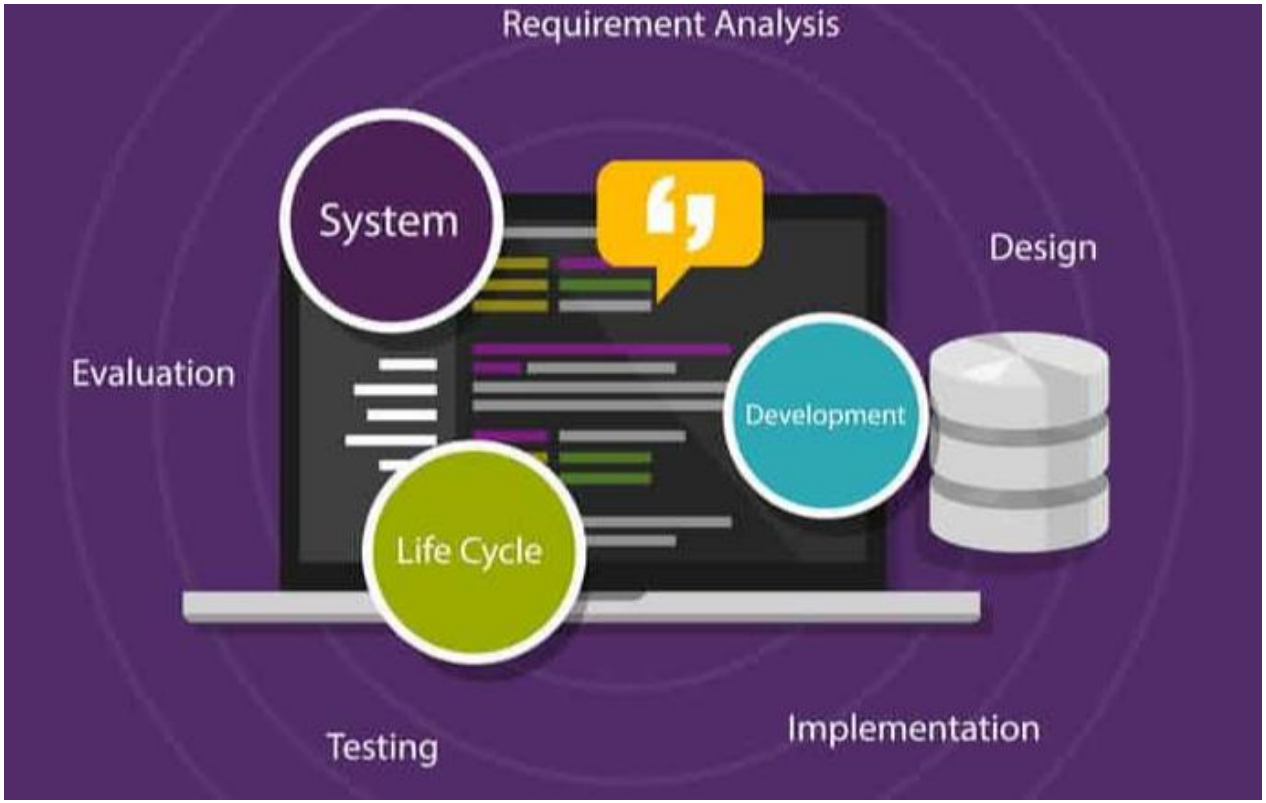
Copyright © March 2026

Requirement Analysis

- Proses ini bertujuan untuk menentukan layanan-layanan (services) yang diperlukan oleh pelanggan dari suatu sistem, serta menetapkan batasan-batasan (constraints) terkait operasional dan pengembangannya.
- Tugas dalam rekayasa perangkat lunak ini berfungsi sebagai penghubung antara alokasi perangkat lunak pada tingkat sistem dan perancangan perangkat lunak

Copyright © March 2026

Requirement Analysis



Sumber : https://businessanalystmentor.com

Copyright © March 2026

Hubungan Requirement Analysis dengan Pengembangan Sistem Informasi

Dalam pengembangan sistem informasi, Requirement Analysis digunakan untuk :

- Mengidentifikasi kebutuhan bisnis dan teknis yang akan diimplementasikan dalam sistem.
- Menghasilkan dokumen spesifikasi kebutuhan yang menjadi acuan bagi tim pengembang.
- Menjadi dasar dalam pengambilan keputusan terkait teknologi, arsitektur, dan desain sistem.

Copyright © March 2026

Jenis Kebutuhan dalam Sistem Informasi dan contohnya

1. Kebutuhan Fungsional

Kebutuhan fungsional adalah spesifikasi yang mendefinisikan bagaimana sistem harus beroperasi dan layanan apa saja yang diberikan kepada pengguna.

Contoh Kebutuhan Fungsional:

- Pengguna dapat melakukan login menggunakan email dan password

Teknik indentifikasi : Dengan menggunakan Use Case Diagram

2. Kebutuhan Non-Fungsional

Kebutuhan non-fungsional mencakup aspek kualitas perangkat lunak yang tidak berkaitan langsung dengan fitur sistem, tetapi lebih ke kinerja dan pengalaman pengguna.

Contoh Kebutuhan Non-Fungsional:

- Sistem harus dapat menangani 1000 transaksi per detik.

Kategori Kebutuhan Non-Fungsional:

1. Keamanan: Proteksi data pengguna, autentikasi, enkripsi.
2. Kinerja: Kecepatan, waktu respons, efisiensi pemrosesan.
3. Keandalan: Ketersediaan sistem (uptime), pemulihan data.
4. Kemudahan Pemeliharaan: Dokumentasi kode, modularitas sistem.

Copyright © March 2026

Teknik Pengumpulan Kebutuhan Sistem

1. Wawancara dengan Pemangku Kepentingan
2. Observasi dan Studi Dokumen
3. Prototyping dan Mockup
4. Use Case dan User Stories

Copyright © March 2026

Dokumentasi Kebutuhan Sistem

1. Software Requirement Specification (SRS)

Dokumen SRS berisi deskripsi rinci tentang kebutuhan sistem, termasuk fungsional dan non-fungsional

2. Teknik Pemodelan Kebutuhan

- Entity-Relationship Diagram (ERD): Untuk memodelkan struktur data.
- Unified Modeling Language (UML): Untuk menggambarkan arsitektur sistem menggunakan diagram seperti Use Case, Activity, dan Sequence Diagram.

Copyright © March 2026

Validasi dan Verifikasi Kebutuhan Sistem

Perlunya Validasi dan verifikasi agar sesuai dengan harapan pemangku kepentingan.

1. Validasi Kebutuhan

- Dilakukan dengan pengguna dan pemangku kepentingan untuk memastikan bahwa kebutuhan yang dikumpulkan sesuai dengan harapan.
- Metode: Review dokumen, prototyping, dan simulasi.

2. Verifikasi Kebutuhan

- Memeriksa konsistensi, kelengkapan, dan kejelasan kebutuhan yang terdokumentasi.
- Menggunakan checklist dan teknik inspeksi formal.

Copyright © March 2026

Studi Kasus

Studi kasus untuk menganalisis kebutuhan suatu sistem informasi misalnya.

- Sistem Informasi Akademik
- Sistem Absensi
- Sistem Ticketing dll

Tahapan yang dilakukan:

1. Mengidentifikasi kebutuhan fungsional dan non-fungsional.
2. Membuat diagram Use Case dan ERD.
3. Menyusun dokumen Software Requirement Specification (SRS).
4. Melakukan validasi kebutuhan dengan pemangku kepentingan.

Copyright © March 2026

Studi Kasus

Tambahan Materi berdasarkan 2 jurnal :

1. Dany, R., Sriyadi, S., & Ginabila, G. (2024). Transformasi sistem electronic assessment management menggunakan metode RAD pada Direktorat Peningkatan Mutu Tenaga Kesehatan. *Jinsan: Jurnal Ilmiah Sistem Informasi dan Teknologi*, 1(1). Diakses dari https://jurnal.bsi.ac.id/index.php/jinsan/article/view/3613/1790
2. Aryasatya, R., Hikmah, N., & Pebrianto, R. (2024). Perancangan aplikasi LAPIM (Laporan Insiden Masyarakat) pada Dinas Perhubungan Pusat Data dan Informasi DKI Jakarta berbasis website. *Jinsan: Jurnal Ilmiah Sistem Informasi dan Teknologi*, 1(1). Diakses dari http://jurnal.bsi.ac.id/index.php/jinsan/article/view/3762/1852

Copyright © March 2026

Case Study

Berdasarkan 2 jurnal diatas, lengkapi

1. Kebutuhan fungsional
2. Kebutuhan diagram Use Case dan ERD/LRS .

	Jurnal 1	Jurnal 2
Judul Artikel Jurnal		
Kebutuhan Fungsional		
Kebutuhan Diagram UML		
Kebutuhan ERD/LRS		

Copyright © March 2026

Evaluasi Kualitas Kebutuhan dalam Pengembangan Perangkat Lunak

- Kelengkapan: Apakah semua kebutuhan utama sudah teridentifikasi?
- Konsistensi: Apakah tidak ada kontradiksi dalam kebutuhan yang didokumentasikan?
- Dapat diverifikasi: Apakah kebutuhan dapat diuji di tahap pengembangan dan pengujian?
- Realistis: Apakah kebutuhan sesuai dengan batasan teknologi dan sumber daya

Link: Handsout Software Requirements Specification

Copyright © March 2026